Автономная некоммерческая организация высшего образования
«Санкт-Петербургский институт экономики и управления»

Кафедра 09.02.07

ПРАКТИЧЕСКАЯ РАБОТА № 4

Дисциплина МДК 02.02
Контракты данных и валидация доступа

Вариант № 2
АИС «Электронная медицинская карта»

Студент: Шваб Роберт Эдуардович
Группа: И13_24_11
Подпись: _____

Руководитель: _____
Дата представления: «___» _____ 2026 г.

Санкт-Петербург
2026 г.

СОДЕРЖАНИЕ

1 ВЫПОЛНЕНИЕ РАБОТЫ.....3
1.1 JSON-контракт запроса.....3
1.2 Правила валидации.....3
1.3 Ответы и аудит.....4
2 ПРОВЕРКА РЕЗУЛЬТАТОВ.....5
ЗАКЛЮЧЕНИЕ.....8
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....9
ПРИЛОЖЕНИЕ А.....10
А.1 Листинг Models.....10
А.2 Листинг AccessPolicy.....10
А.3 Листинг TokenAuthentication.....11
А.4 Листинг GatewayFactory.....12
А.5 Листинг EncryptedAuditStore.....14
А.6 Листинг ConsoleChecks.....15
А.7 Листинг Program.....16

1 ВЫПОЛНЕНИЕ РАБОТЫ

Цель работы — спроектировать и реализовать JSON-запрос проверки доступа к ЭМК, правила валидации и ответы сервера. Работа выполнена по варианту 2 нового сборника [2].

В примерах используются вымышленные пациенты. Оформление выполнено по требованиям института [1].

1.1 JSON-контракт запроса

```
{
  "requestId": "33333333-3333-4333-8333-333333333333",
  "patientId": "11111111-1111-4111-8111-111111111111",
  "purpose": "Treatment",
  "workstationId": "PC-001"
}
```

Запрос отправляется POST /api/v1/access/check с Content-Type application/json и Bearer-токеном в заголовке. ActorId и роль отсутствуют в теле: их устанавливает сервер. Неизвестные поля JSON отклоняются, чтобы клиент не мог передать себе дополнительные полномочия [3].

1.2 Правила валидации

Сведения представлены в таблице 1.

Таблица 1 — Проверки запроса

Поле или условие	Требование	Ошибка
requestId	Непустой UUID; повтор с другой операцией запрещён	400 / 409
patientId	Непустой UUID; назначение сотруднику	400 / 403
purpose	Treatment или NursingCare; соответствует роли	400 / 403
workstationId	Regex: \A[A-Z]{2}-[0-9]{3}\z	400
Токен	Известен текущему серверу	401
Doctor / Nurse	Нужна назначенная карта и подходящая цель	403
Тело запроса	JSON, не null, без лишних полей	400 / 415

Источник: составлено по учебному заданию [2].

Admin управляет конфигурацией, но не получает медицинский доступ автоматически. Идентификатор станции проверяет формат параметра и не

заменяет аутентификацию устройства. Повтор того же requestId с теми же данными возвращает прежнее решение без повторного события аудита.

1.3 Ответы и аудит

Сведения представлены в таблице 2.

Таблица 2 — Ответы сервера

HTTP	Code / результат	Смысл
200	decision=allow, scope=summary	Доступ разрешён
400	validation_error / invalid_json	Неверные поля или типы
401	unauthorized	Нет действующего токена
403	access_denied	Роль, назначение или цель не подходят
404	not_found	Назначенная карта отсутствует
409	request_id_conflict	UUID использован с другой операцией
415	unsupported_media_type	Передан не JSON

Источник: составлено по учебному заданию [2].

В файл аудита записывается шифротекст AES-256-GCM с новым 12-байтовым nonce и 16-байтовым тегом. Ключ — 32 случайных байта — хранится в памяти учебного процесса; его нет в репозитории. Подмена тега обнаруживается при расшифровании [4].

2 ПРОВЕРКА РЕЗУЛЬТАТОВ

Запущен реальный ASP.NET Core API и выполнены HTTP-запросы. Назначенному врачу выдан 200; Admin получил 403; отсутствие токена — 401; неверная станция — 400.

dotnet run --project src/AIS.Gateway -- --console-check --scenario all

Результат показан на рисунке 1.

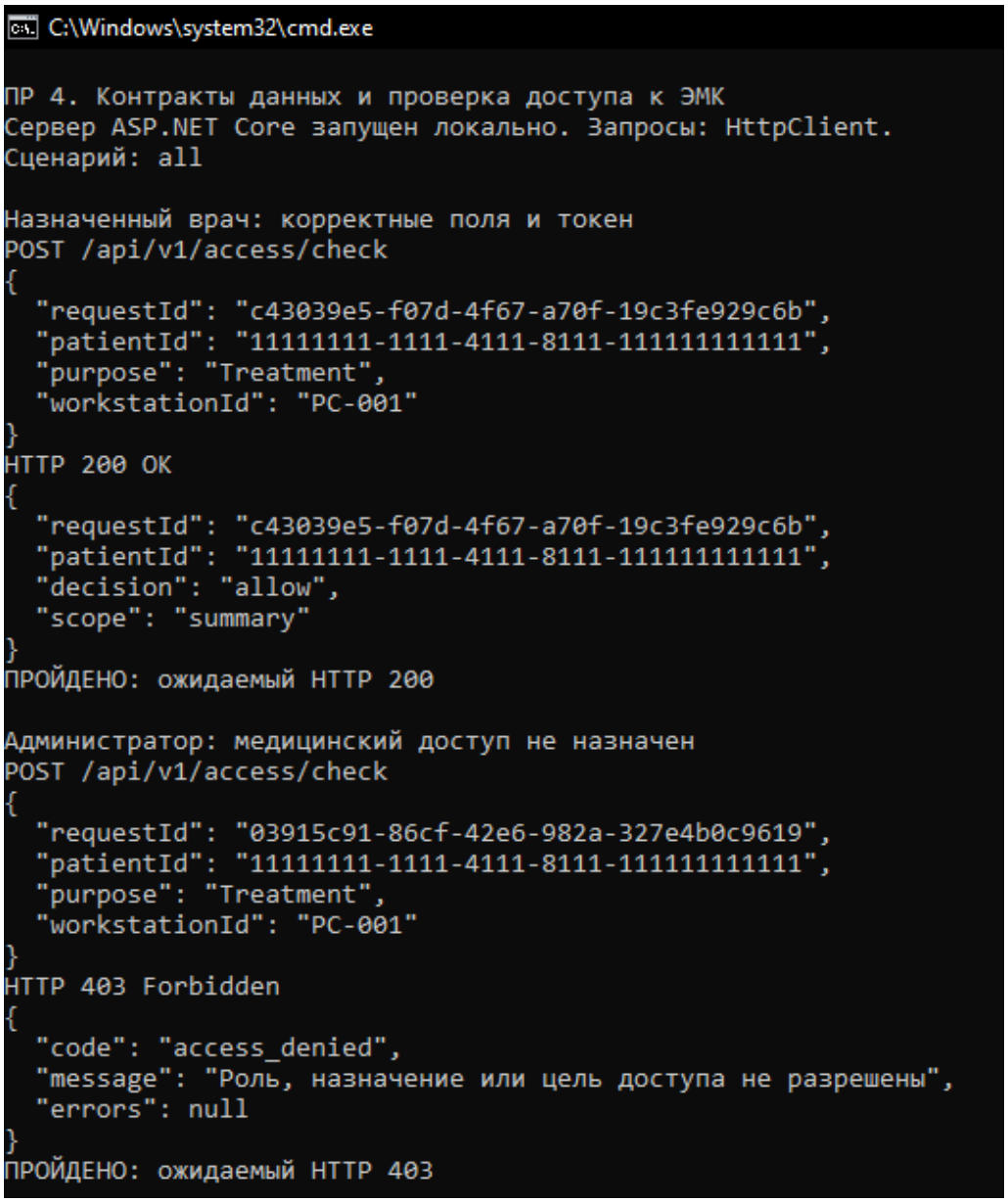


Рисунок 1 — Разрешение врачу и отказ роли Admin: HTTP 200 и 403

Источник: снимок консольного запуска программы C# в Windows.

Результат показан на рисунке 2.

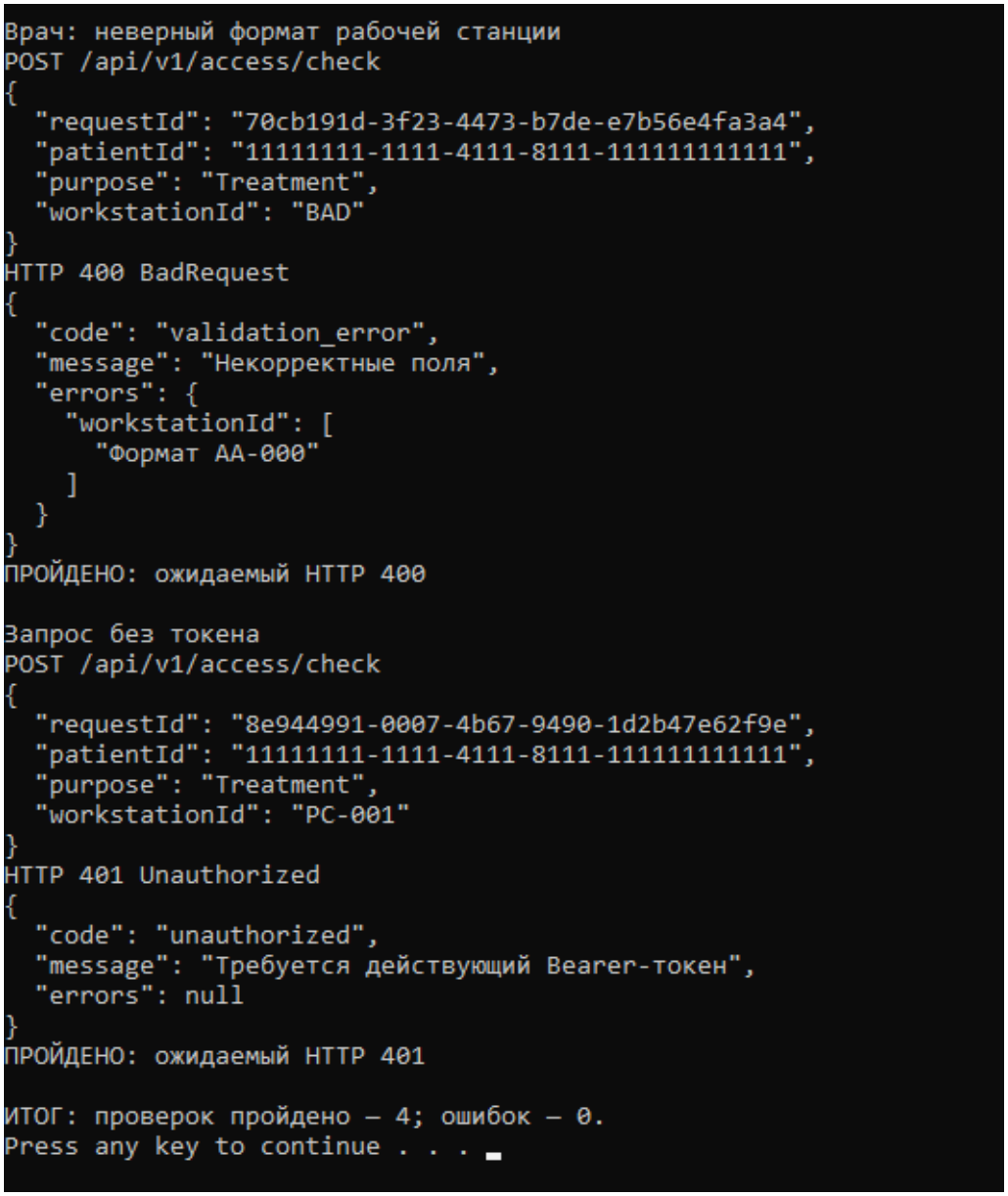


Рисунок 2 — Ошибки формата станции и отсутствия токена: HTTP 400 и 401

Источник: снимок консольного запуска программы C# в Windows.

Проверены также null JSON, неверный UUID, лишнее поле role, повтор requestId и целостность шифрованного аудита. Листинги контрактов, правил и серверного обработчика приведены в приложении А.

Проверки выполнены в Visual Studio на Windows консольным проектом AIS.Tests: TOTAL PASS=52; FAIL=0, процесс завершился с кодом 0. Проверены архитектура, mapping, SQLite, AES-GCM, API и MVVM. AIS.Tests — собственный консольный запускатель, не xUnit.

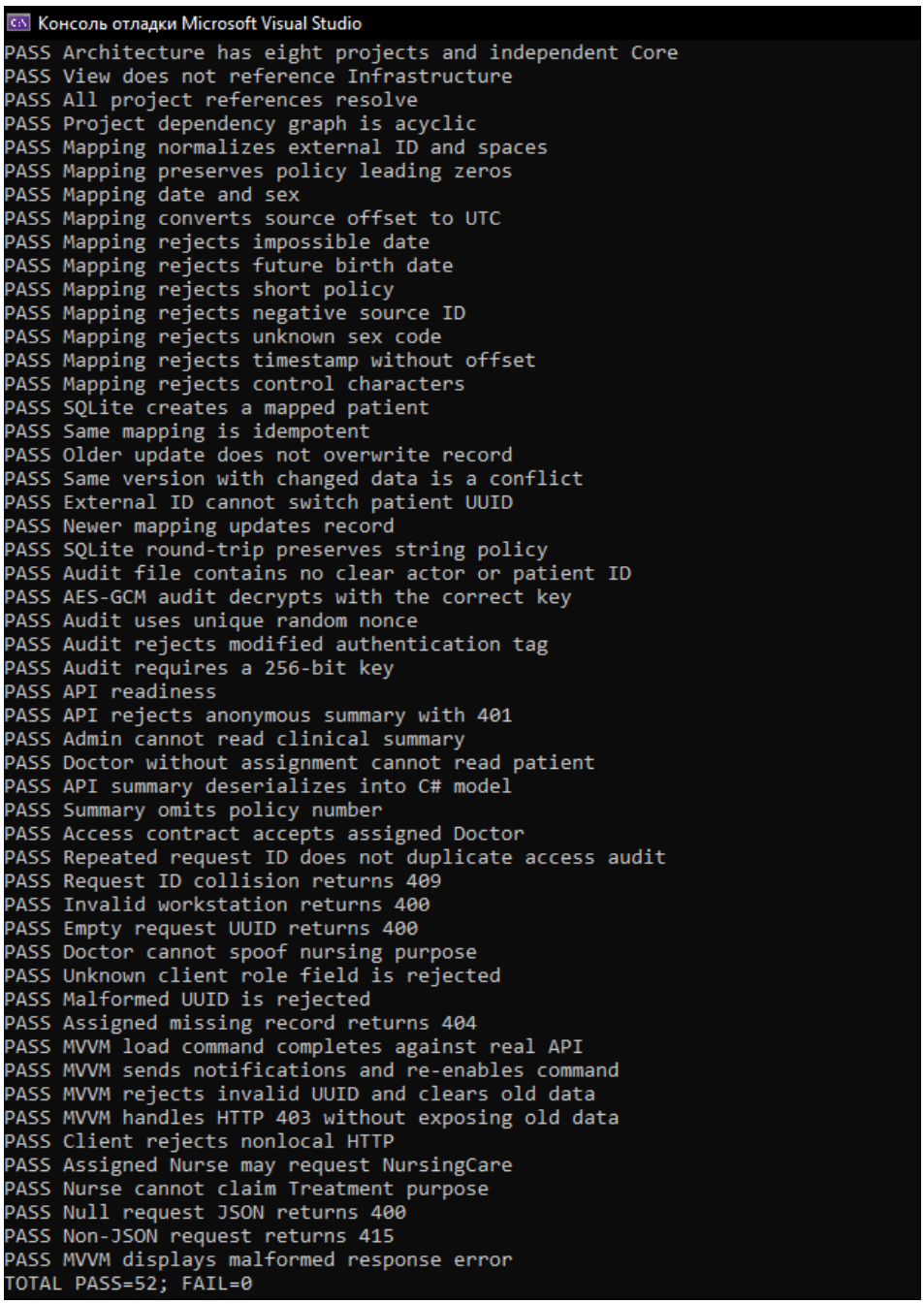


Рисунок 3 — Результат запуска в Visual Studio Windows

Источник: снимок выполнения, предоставленный автором 12.11.2026.

Исходники и материалы актуальной публикации: https://github.com/rob4ik72-del/MDK0202-EMR

ЗАКЛЮЧЕНИЕ

Сформирован защищённый контракт проверки доступа и реализован обработчик на C#. Синтаксические ошибки отделены от отказа в полномочиях.

Запрос не определяет собственную роль. Клинический доступ ограничен назначением, а аудит не сохраняет идентификаторы в открытом виде.

Репозиторий проекта: https://github.com/rob4ik72-del/MDK0202-EMR

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. СПбИЭУ. Положение о требованиях по подготовке и оформлению письменных работ и приложение М: локальный нормативный акт, предоставленный в материалах обучения. [1].

2. МДК 02.02. Детализированный сборник практических работ ПР1–ПР9: электронный учебный материал; вариант 2 «Электронная медицинская карта». [2].

3. OpenAPI Initiative. OpenAPI Specification 3.0.3. URL: https://spec.openapis.org/oas/v3.0.3 (дата обращения: 05.11.2026) [3].

4. Microsoft Learn. Cross-platform cryptography in .NET. URL: https://learn.microsoft.com/en-us/dotnet/standard/security/cross-platform-cryptography (дата обращения: 05.11.2026) [4].

ПРИЛОЖЕНИЕ А

А.1 Листинг Models

src/AIS.Core/Models.cs
```
namespace Ais.Core;
public sealed record SourcePatient(string ExternalId, string FullName, string BirthDate, string Sex, string
PolicyNumber, string UpdatedAt);
public sealed record Patient(int ExternalId, Guid Id, string FullName, DateOnly BirthDate, string Sex, string
PolicyNumber, DateTimeOffset UpdatedAt);
public sealed record PatientSummary(Guid Id, string FullName, string BirthDate, string Sex, string Note);
public sealed record AccessRequest(Guid RequestId, Guid PatientId, string Purpose, string WorkstationId);
public sealed record AccessReply(Guid RequestId, Guid PatientId, string Decision, string Scope);
public sealed record ApiError(string Code, string Message, Dictionary<string, string[]>? Errors = null);
public sealed record Actor(string Id, string Role, Guid[] AssignedPatients);
public interface IPatientRepository
{
   Patient? Find(Guid id);
   string Upsert(Patient patient);
}

public interface IAuditStore
{
   void Write(string actor, Guid patient, string action, string result);
}

public interface IMedicalApiClient
{
   Task<PatientSummary> GetSummaryAsync(Guid id, CancellationToken ct = default);
   Task<AccessReply> CheckAccessAsync(AccessRequest request, CancellationToken ct = default);
}

public static class SampleData
{
   public static readonly Guid PatientId = Guid.Parse("11111111-1111-4111-8111-111111111111");
   public static Patient Patient => new(42, PatientId, "Пациент Учебный Один", new DateOnly(1990, 2, 14),
"F", "0012345678901234", DateTimeOffset.Parse("2026-10-20T10:00:00+03:00"));
   public static SourcePatient Source => new("000042", "  Пациент   Учебный Один ", "14.02.1990", "Ж",
"0012345678901234", "2026-10-20T10:00:00+03:00");
}
```

А.2 Листинг AccessPolicy

src/AIS.Core/AccessPolicy.cs
```
using System.Text.RegularExpressions;

namespace Ais.Core;
public static class AccessPolicy
{
   public static Dictionary<string, string[]> Validate(AccessRequest r)
   {
      var errors = new Dictionary<string, string[]>();
      if (r.RequestId == Guid.Empty)
         errors["requestId"] = new[]
```

```
            {
                "Нужен непустой UUID"
            };
        if (r.PatientId == Guid.Empty)
            errors["patientId"] = new[]
            {
                "Нужен непустой UUID"
            };
        if (r.Purpose is not ("Treatment" or "NursingCare"))
            errors["purpose"] = new[]
            {
                "Treatment или NursingCare"
            };
        if (!Regex.IsMatch(r.WorkstationId ?? "", @"\A[A-Z]{2}-[0-9]{3}\z"))
            errors["workstationId"] = new[]
            {
                "Формат AA-000"
            };
        return errors;
    }

    public static bool CanRead(Actor actor, Guid patient) => (actor.Role is "Doctor" or "Nurse") &&
actor.AssignedPatients.Contains(patient);
    public static bool CanCheck(Actor actor, AccessRequest r) => CanRead(actor, r.PatientId) && ((actor.Role ==
"Doctor" && r.Purpose == "Treatment") || (actor.Role == "Nurse" && r.Purpose == "NursingCare"));
}
```

A.3 Листинг TokenAuthentication

```
src/AIS.Gateway/TokenAuthentication.cs
using Microsoft.AspNetCore.Http;
using System.Security.Claims;
using System.Security.Cryptography;
using System.Text;
using System.Text.Encodings.Web;
using Microsoft.AspNetCore.Authentication;
using Microsoft.Extensions.Logging;
using Microsoft.Extensions.Options;
using Ais.Core;

namespace Ais.Gateway;
public sealed class TokenDirectory
{
    readonly Dictionary<string, Actor> actors = new();
    public string Issue(Actor actor)
    {
        string token = Convert.ToHexString(RandomNumberGenerator.GetBytes(32));
        actors[Fingerprint(token)] = actor;
        return token;
    }

    static string Fingerprint(string s) => Convert.ToHexString(SHA256.HashData(Encoding.UTF8.GetBytes(s)));
    public Actor? Resolve(string token) => actors.GetValueOrDefault(Fingerprint(token));
}
```

```
public sealed class TokenAuthentication(IOptionsMonitor<AuthenticationSchemeOptions> options,
ILoggerFactory logger, UrlEncoder encoder, TokenDirectory tokens) :
AuthenticationHandler<AuthenticationSchemeOptions>(options, logger, encoder)
{
   protected override Task<AuthenticateResult> HandleAuthenticateAsync()
   {
      string header = Request.Headers.Authorization.ToString();
      if (!header.StartsWith("Bearer ", StringComparison.OrdinalIgnoreCase))
         return Task.FromResult(AuthenticateResult.NoResult());
      var actor = tokens.Resolve(header[7..]);
      if (actor == null)
         return Task.FromResult(AuthenticateResult.Fail("Invalid token"));
      var claims = new[]
      {
         new Claim(ClaimTypes.NameIdentifier, actor.Id),
         new Claim(ClaimTypes.Role, actor.Role)
      };
      return Task.FromResult(AuthenticateResult.Success(new AuthenticationTicket(new ClaimsPrincipal(new
ClaimsIdentity(claims, Scheme.Name)), Scheme.Name)));
   }

   protected override async Task HandleChallengeAsync(AuthenticationProperties properties)
   {
      Response.StatusCode = 401;
      Response.Headers.WWWAuthenticate = "Bearer";
      await Response.WriteAsJsonAsync(new ApiError("unauthorized", "Требуется действующий Bearer-
токен"));
   }

   protected override async Task HandleForbiddenAsync(AuthenticationProperties properties)
   {
      Response.StatusCode = 403;
      await Response.WriteAsJsonAsync(new ApiError("access_denied", "Роль не разрешает операцию"));
   }
}
```

A.4 Листинг GatewayFactory

```
src/AIS.Gateway/GatewayFactory.cs
using System.Security.Claims;
using System.Text.Json;
using System.Text.Json.Serialization;
using Microsoft.AspNetCore.Authentication;
using Microsoft.AspNetCore.Builder;
using Microsoft.AspNetCore.Http;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;
using Ais.Core;

namespace Ais.Gateway;
public static class GatewayFactory
{
   public static WebApplication Create(IPatientRepository repository, IAuditStore audit, TokenDirectory
tokens, string[]? args = null)
   {
      var builder = WebApplication.CreateBuilder(args ?? Array.Empty<string>());
```

```
builder.Logging.ClearProviders();
builder.Services.AddSingleton(tokens);
builder.Services.AddAuthentication("Bearer").AddScheme<AuthenticationSchemeOptions,
TokenAuthentication>("Bearer", _ =>
{
});
builder.Services.AddAuthorization();
builder.Services.ConfigureHttpJsonOptions(o => o.SerializerOptions.UnmappedMemberHandling =
JsonUnmappedMemberHandling.Disallow);
var app = builder.Build();
app.UseAuthentication();
app.UseAuthorization();
var requests = new Dictionary<Guid, (string Actor, AccessRequest Request, AccessReply Reply)>();
var gate = new object ();
Actor ActorFor(HttpContext c) => tokens.Resolve(c.Request.Headers.Authorization.ToString()[7..])!;
IResult Error(int status, string code, string message) => Results.Json(new ApiError(code, message),
statusCode: status);
app.MapGet("/health", () => Results.Ok(new { status = "ready" }));
app.MapGet("/api/v1/patients/{id:guid}/summary", (Guid id, HttpContext context) =>
{
   var actor = ActorFor(context);
   if (!AccessPolicy.CanRead(actor, id))
   {
      audit.Write(actor.Id, id, "read_summary", "denied");
      return Error(403, "access_denied", "Нет доступа к карте");
   }

   var patient = repository.Find(id);
   if (patient == null)
      return Error(404, "not_found", "Карта не найдена");
   audit.Write(actor.Id, id, "read_summary", "allowed");
   return Results.Ok(new PatientSummary(patient.Id, patient.FullName,
patient.BirthDate.ToString("yyyy-MM-dd"), patient.Sex, "Учебная запись без медицинского содержания"));
}).RequireAuthorization();
app.MapPost("/api/v1/access/check", async (HttpContext context) =>
{
   if (!context.Request.HasJsonContentType())
      return Error(415, "unsupported_media_type", "Нужен Content-Type application/json");
   AccessRequest? parsed;
   try
   {
      parsed = await context.Request.ReadFromJsonAsync<AccessRequest>();
   }
   catch (JsonException)
   {
      return Error(400, "invalid_json", "Неверный JSON или тип поля");
   }
   catch (BadHttpRequestException)
   {
      return Error(400, "invalid_json", "Невозможно прочитать запрос");
   }

   if (parsed == null)
      return Error(400, "invalid_json", "Тело запроса не должно быть null");
   AccessRequest request = parsed;
   var errors = AccessPolicy.Validate(request);
   if (errors.Count > 0)
```

```
            return Results.Json(new ApiError("validation_error", "Некорректные поля", errors), statusCode:
400);
        var actor = ActorFor(context);
        if (!AccessPolicy.CanCheck(actor, request))
        {
            audit.Write(actor.Id, request.PatientId, "check_access", "denied");
            return Error(403, "access_denied", "Роль, назначение или цель доступа не разрешены");
        }

        if (repository.Find(request.PatientId) == null)
            return Error(404, "not_found", "Карта не найдена");
        lock (gate)
        {
            if (requests.TryGetValue(request.RequestId, out var old))
                return old.Actor == actor.Id && old.Request == request ? Results.Ok(old.Reply) : Error(409,
"request_id_conflict", "UUID уже использован с другими параметрами");
            var reply = new AccessReply(request.RequestId, request.PatientId, "allow", "summary");
            audit.Write(actor.Id, request.PatientId, "check_access", "allowed");
            requests.Add(request.RequestId, (actor.Id, request, reply));
            return Results.Ok(reply);
        }
    }).RequireAuthorization();
    return app;
  }
}
```

A.5 Листинг EncryptedAuditStore

```
src/AIS.Infrastructure/EncryptedAuditStore.cs
using System.Security.Cryptography;
using System.Text;
using System.Text.Json;
using Ais.Core;

namespace Ais.Infrastructure;
public sealed record AuditEnvelope(string Nonce, string Ciphertext, string Tag);
public sealed class EncryptedAuditStore(string path, byte[] key) : IAuditStore
{
  readonly byte[] secret = key.Length == 32 ? key.ToArray() : throw new ArgumentException("Ключ AES-256
должен иметь 32 байта");
  readonly object gate = new();
  static readonly byte[] Aad = Encoding.UTF8.GetBytes("EMR-AUDIT-v1");
  public void Write(string actor, Guid patient, string action, string result)
  {
    byte[] plain = JsonSerializer.SerializeToUtf8Bytes(new { actor, patient, action, result, at =
DateTimeOffset.UtcNow });
    var nonce = RandomNumberGenerator.GetBytes(12);
    var cipher = new byte[plain.Length];
    var tag = new byte[16];
    using var aes = new AesGcm(secret, 16);
    aes.Encrypt(nonce, plain, cipher, tag, Aad);
    var envelope = new AuditEnvelope(Convert.ToBase64String(nonce), Convert.ToBase64String(cipher),
Convert.ToBase64String(tag));
    lock (gate)
      File.AppendAllText(path, JsonSerializer.Serialize(envelope) + Environment.NewLine);
    CryptographicOperations.ZeroMemory(plain);
  }
```

```
    public string DecryptForVerification(string line)
    {
        var e = JsonSerializer.Deserialize<AuditEnvelope>(line)!;
        var bytes = Convert.FromBase64String(e.Ciphertext);
        var plain = new byte[bytes.Length];
        using var aes = new AesGcm(secret, 16);
        aes.Decrypt(Convert.FromBase64String(e.Nonce), bytes, Convert.FromBase64String(e.Tag), plain, Aad);
        return Encoding.UTF8.GetString(plain);
    }
}
```

A.6 Листинг ConsoleChecks

src/AIS.Gateway/ConsoleChecks.cs

```
using System.Net;
using System.Net.Http.Headers;
using System.Text;
using System.Text.Json;
using Ais.Core;
using Ais.Infrastructure;
using Microsoft.AspNetCore.Hosting.Server;
using Microsoft.AspNetCore.Hosting.Server.Features;
using Microsoft.Extensions.DependencyInjection;

namespace Ais.Gateway;

public static class ConsoleChecks
{
    public static async Task<int> RunAsync(string scenario)
    {
        string directory = Path.Combine(Path.GetTempPath(), "EmrConsole_" + Guid.NewGuid().ToString("N"));
        Directory.CreateDirectory(directory);
        try
        {
            var repository = new SqlitePatientRepository(Path.Combine(directory, "patients.db"));
            repository.Upsert(SampleData.Patient);
            var audit = new EncryptedAuditStore(Path.Combine(directory, "audit.enc"),
System.Security.Cryptography.RandomNumberGenerator.GetBytes(32));
            var tokens = new TokenDirectory();
            string doctor = tokens.Issue(new Actor("doctor-console", "Doctor", new[] { SampleData.PatientId }));
            string admin = tokens.Issue(new Actor("admin-console", "Admin", Array.Empty<Guid>()));
            await using var app = GatewayFactory.Create(repository, audit, tokens, new[] { "--urls",
"http://127.0.0.1:0" });
            await app.StartAsync();
            string address =
app.Services.GetRequiredService<IServer>().Features.Get<IServerAddressesFeature>()!.Addresses.Single();
            using var client = new HttpClient { BaseAddress = new Uri(address) };
            int count = 0;
            var jsonOptions = new JsonSerializerOptions { WriteIndented = true, Encoder =
System.Text.Encodings.Web.JavaScriptEncoder.UnsafeRelaxedJsonEscaping };
            Console.WriteLine("ПР 4. Контракты данных и проверка доступа к ЭМК");
            Console.WriteLine("Сервер ASP.NET Core запущен локально. Запросы: HttpClient.");
            Console.WriteLine("Сценарий: " + scenario);
            Console.WriteLine();
            async Task Send(string name, string? token, string json, HttpStatusCode expected)
```

```
        {
            using var request = new HttpRequestMessage(HttpMethod.Post, "/api/v1/access/check");
            if (token != null) request.Headers.Authorization = new AuthenticationHeaderValue("Bearer", token);
            request.Content = new StringContent(json, Encoding.UTF8, "application/json");
            Console.WriteLine(name);
            Console.WriteLine("POST /api/v1/access/check");
            using (var body = JsonDocument.Parse(json)) Console.WriteLine(JsonSerializer.Serialize(body,
jsonOptions));
            using var response = await client.SendAsync(request);
            string result = await response.Content.ReadAsStringAsync();
            Console.WriteLine("HTTP " + (int)response.StatusCode + " " + response.StatusCode);
            using (var body = JsonDocument.Parse(result)) Console.WriteLine(JsonSerializer.Serialize(body,
jsonOptions));
            if (response.StatusCode != expected) throw new InvalidOperationException("Неожиданный статус:
" + response.StatusCode);
            Console.WriteLine("ПРОЙДЕНО: ожидаемый HTTP " + (int)expected);
            Console.WriteLine();
            count++;
        }
        string Payload(string station = "PC-001", string purpose = "Treatment") => JsonSerializer.Serialize(new
AccessRequest(Guid.NewGuid(), SampleData.PatientId, purpose, station), new
JsonSerializerOptions(JsonSerializerDefaults.Web));
        if (scenario is "success" or "all") await Send("Назначенный врач: корректные поля и токен", doctor,
Payload(), HttpStatusCode.OK);
        if (scenario is "denied" or "all") await Send("Администратор: медицинский доступ не назначен",
admin, Payload(), HttpStatusCode.Forbidden);
        if (scenario is "validation" or "all") await Send("Врач: неверный формат рабочей станции", doctor,
Payload("BAD"), HttpStatusCode.BadRequest);
        if (scenario is "unauthorized" or "all") await Send("Запрос без токена", null, Payload(),
HttpStatusCode.Unauthorized);
        if (count == 0) throw new ArgumentException("Сценарии: success, denied, validation, unauthorized,
all");
        Console.WriteLine("ИТОГ: проверок пройдено — " + count + "; ошибок — 0.");
        await app.StopAsync();
        return 0;
    }
    finally
    {
        try { Directory.Delete(directory, true); } catch (IOException) { }
    }
  }
}
```

A.7 Листинг Program

src/AIS.Gateway/Program.cs

```
using System.Security.Cryptography;
using Ais.Core;
using Ais.Infrastructure;
using Ais.Gateway;

if (args.Contains("--console-check"))
{
  int index = Array.IndexOf(args, "--scenario");
  string scenario = index >= 0 && index + 1 < args.Length ? args[index + 1] : "all";
  Environment.ExitCode = await ConsoleChecks.RunAsync(scenario);
  return;
```

```
}

if (!args.Contains("--demo"))
    throw new InvalidOperationException("Запуск учебного сервера: --demo. Для рабочего контура
нужны отдельные OIDC/TLS и управление ключами.");
var folder = Path.Combine(Path.GetTempPath(), "EmrDemo_" + Guid.NewGuid().ToString("N"));
Directory.CreateDirectory(folder);
var repository = new SqlitePatientRepository(Path.Combine(folder, "patients.db"));
repository.Upsert(SampleData.Patient);
var audit = new EncryptedAuditStore(Path.Combine(folder, "audit.enc"),
RandomNumberGenerator.GetBytes(32));
var tokens = new TokenDirectory();
string doctor = tokens.Issue(new Actor("doctor-demo", "Doctor", new[] { SampleData.PatientId }));
string admin = tokens.Issue(new Actor("admin-demo", "Admin", Array.Empty<Guid>()));
string nurse = tokens.Issue(new Actor("nurse-demo", "Nurse", new[] { SampleData.PatientId }));
var app = GatewayFactory.Create(repository, audit, tokens, new[] { "--urls", "http://127.0.0.1:5274" });
DemoViews.Map(app, doctor, admin, nurse);
Console.WriteLine("Локальный API: http://127.0.0.1:5274");
Console.WriteLine("Doctor token (только для этого учебного запуска): " + doctor);
Console.WriteLine("Nurse token (учебный): " + nurse);
try
{
    await app.RunAsync();
}
finally
{
    await app.DisposeAsync();
    try
    {
        Directory.Delete(folder, true);
    }
    catch (IOException)
    {
    }
}
```